

BF

DOCUMENTO TÉCNICO DE ARQUITETURA

BodyFlow versão WhatsApp

Stack, componentes, dados, integrações e fluxos ponta a ponta da plataforma quando o paciente usava somente o WhatsApp.

Recorte histórico

Este documento descreve o Agente MPP como produto conversacional WhatsApp. O painel web administrativo faz parte da operação. App iOS, API mobile, APNs, StoreKit, autenticação de paciente por e-mail e demais elementos app-first estão fora do escopo.

Base técnica

Git `origin/main`

SHA `280b3d7b569a`

18 de julho de 2026

Fontes

Código, migrations, ADRs, runbooks e auditoria técnica do repositório. Nenhum secret ou dado pessoal foi incluído.

Sumário

1. Escopo e premissas
2. Produto e capacidades
3. Arquitetura de contexto
4. Stack tecnológico
5. Organização do monorepo
6. Fluxo de entrada WhatsApp
7. Fluxos de áudio e imagem
8. Pipeline do agente
9. Confirmação e escrita
10. Fluxo de resposta
11. Motor determinístico
12. Arquitetura de dados
13. Workers e automações
14. Painel administrativo
15. Assinaturas e billing
16. Observabilidade e recuperação
17. Trust boundaries e segurança
18. DevOps e deploy
19. Redundâncias e modos de falha
20. Limitações históricas
21. Evidências e glossário

Escopo e premissas

Na fase documentada, o WhatsApp era a interface completa do paciente. Conversa, onboarding, registros, confirmações, lembretes, áudio, fotos, reavaliação e retorno do agente aconteciam dentro do canal oficial da Meta.

Incluído

- WhatsApp Cloud API como único canal de paciente em produção.
- Supabase Postgres, Edge Functions, Auth administrativo, pgvector, pg_cron e pg_net.
- Inngest como orquestrador durável de mensagens, crons e retentativas.
- OpenRouter, Groq, ElevenLabs, Cartesia e Helicone opcional.
- Next.js administrativo na Vercel e endpoints serverless.
- Stripe para assinatura, Telegram para operação interna e GitHub para versionamento.

Explicitamente excluído

- Aplicativo iOS, SwiftUI e projeto Xcode.
- API/BFF mobile versionada e contratos próprios do app.
- APNs, push nativo e deep links mobile.
- StoreKit, App Store e compra dentro do aplicativo.
- Autenticação de paciente por e-mail ou Apple.
- Storage de mídia desenhado para um app nativo.

Identidade do paciente naquela fase

O paciente não possuía conta Supabase Auth. Sua identidade de domínio era o número WhatsApp normalizado em E.164, persistido em `users.wpp`. Supabase Auth atendia os administradores do painel, não os pacientes.

A referência técnica principal deste relatório é o snapshot Git `280b3d7b569a48410207a901182e003e436f2aef`. Documentações mais antigas do repositório exibem contagens anteriores de migrations e workers; as contagens deste PDF foram recalculadas no snapshot indicado.

O que a versão WhatsApp entregava

Onboarding

Coleta conversacional de perfil, objetivo, protocolo, localização, preferências e rotina. Botões para até 3 opções e listas para até 10.

Registro multimodal

Refeições e treinos por texto, áudio, foto e legenda. Fotos também cobriam balança, corpo e rótulos.

Saldo diário

Card canônico com consumido, meta, proteína, exercício e Bloco 7700, calculado fora da LLM.

Confirmação

Propostas de refeição podiam aguardar o botão “Sim, registrar” ou “Editar” antes da escrita definitiva.

Coaching

Comentários educativos, lembretes de refeição, engajamento, reavaliação, dieta e treino sob demanda.

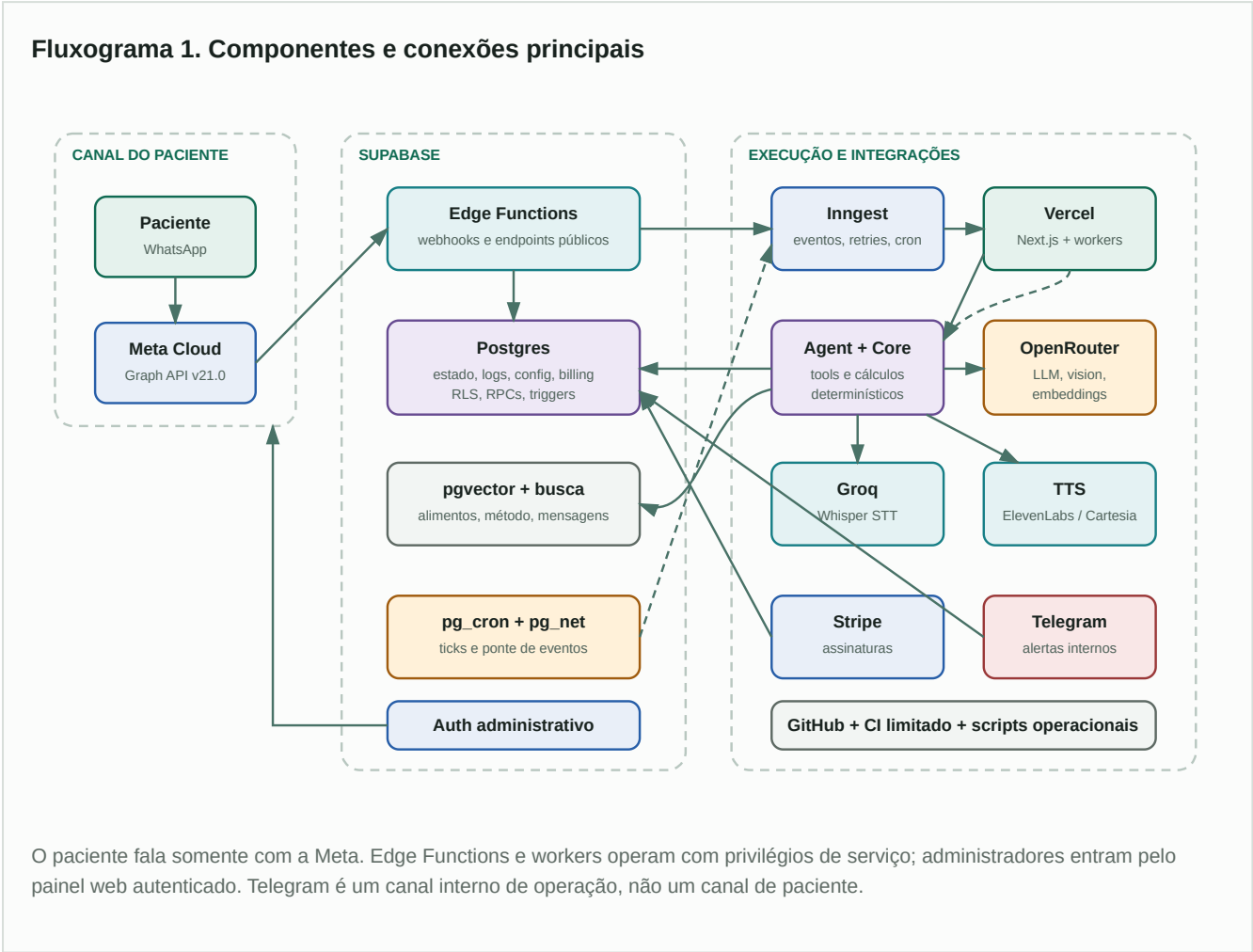
Operação

Painel web para usuários, conversas, prompts, alimentos, auditoria, crons, custos, billing e intervenções.

Princípio de desenho

A LLM interpretava linguagem, mantinha a persona e escolhia ferramentas. O código e o banco eram responsáveis por cálculos, validação, idempotência e escrita. O card final era reconstruído a partir do estado persistido.

Arquitetura de contexto



Planos da arquitetura

PLANO	RESPONSABILIDADE	COMPONENTES CENTRAIS
Experiência	Conversa, botões, mídia, reações, lembretes e cards.	WhatsApp Cloud, provider de mensageria, templates.
Orquestração	Ordenação por usuário, debounce, retries, espera e execução agendada.	Inngest, <code>message_buffer</code> , outbox, <code>pg_cron</code> .
Inteligência	Interpretação, persona, multimodal, RAG e escolha de tools.	<code>@mpp/agent</code> , OpenRouter, Groq, providers.
Domínio	Cálculos, invariantes, metas, balanço, Bloco 7700 e registros atômicos.	<code>@mpp/core</code> , tools, RPCs Postgres.
Dados	Estado do paciente, histórico, configuração, billing, auditoria e vetores.	Supabase Postgres, RLS, triggers, views, pgvector.

PLANO	RESPONSABILIDADE	COMPONENTES CENTRAIS
Operação	Administração, monitoramento, qualidade, deploy e intervenção humana.	Next.js/Vercel, Telegram, GitHub, scripts.

Stack por camada

CAMADA	TECNOLOGIA / SERVIÇO	USO NO SISTEMA
Runtime	Node.js 22+, TypeScript 5.7, ESM	Aplicação, packages, workers, scripts e avaliações.
Monorepo	pnpm 10.33.2 + Turborepo 2.5	Workspaces, execução filtrada, builds e testes.
Qualidade	Vitest 2.1.8 + Biome 2.2.5	Testes de domínio, regressões, golden cases, lint e formato.
Admin web	Next.js 15.1, React 19, Tailwind, Radix UI	Painel administrativo, server actions e rotas API.
Hosting web	Vercel	Admin, <code>/api/inngest</code> , mídia proxy e checkout.
Canal	WhatsApp Cloud API, Graph v21.0	Texto, áudio, imagem, botões, listas, templates, reações e status.
Banco	Supabase Postgres	Dados de domínio, configuração, observabilidade, billing e operações atômicas.
Banco avançado	pgvector, pg_trgm, unaccent, pg_cron, pg_net, pgcrypto	Busca semântica/fuzzy, normalização, agendas e UUIDs.
Edge	Supabase Edge Functions, Deno	Webhooks Meta/Stripe/Telegram e auditoria.
Orquestração	Inngest 3.27	Eventos duráveis, concorrência, retries, passos e crons.
LLM	OpenRouter via SDK OpenAI	Conversa, tool calling, geração, comentários e judge.
Vision	OpenRouter multimodal	Refeição, corpo, balança, rótulo nutricional e classificação.
Embeddings	OpenRouter, 1024 dimensões	RAG do método e busca semântica.
STT	Groq Whisper	Transcrição de áudio em português.
TTS	ElevenLabs + Cartesia	Voz de âncora e voz operacional, com roteamento.
Billing	Stripe	Checkout, assinatura, renovação, inadimplência e cancelamento.
Operação interna	Telegram	Alertas, relatórios e aprovações humanas.
Observabilidade LLM	Helicone opcional + eventos próprios	Custos, tokens, cache, latência e rastreamento.
Versionamento	GitHub	Código, PRs, histórico, workflow preventivo e releases manuais.

Modelos eram configuração de runtime

Os nomes de modelo não eram uma constante arquitetural. `agent_configs` , `agent_rules` , `global_config` e loaders de runtime podiam alterar modelo, temperatura, tokens, vision e comportamento sem redesenhar o fluxo. No baseline, a família Anthropic era usada nos caminhos principais e Haiku nos turnos mais baratos; OpenRouter era o gateway comum.

Organização do monorepo

```
agentmpp/
├─ apps/
│  ├─ admin/           Next.js, painel e endpoints serverless
│  └─ cli/             chat local de teste
├─ packages/
│  ├─ core/            domínio e fórmulas determinísticas
│  ├─ agent/           pipeline, tools, guards e composição
│  ├─ providers/       Meta, OpenRouter, Groq e TTS
│  ├─ db/              cliente Supabase e tipos gerados
│  └─ ingest-functions/ workers, crons e recuperação
├─ supabase/
│  ├─ functions/       6 Edge Functions
│  ├─ migrations/     114 migrations no baseline
│  └─ tests/           testes SQL de RPCs e invariantes
├─ scripts/           seed, auditoria, reparo e deploy
├─ eval/              casos e runner de avaliação
└─ docs/              ADRs, runbooks, cálculo e especificações
```

PACOTE	RESPONSABILIDADE	NÃO DEVERIA POSSUIR
@mpp/core	Funções puras de nutrição, metas, balanço, protocolo e bloco.	I/O, chamadas LLM ou mensageria.
@mpp/agent	Contexto, prompts, tool loop, guardrails, composição e persistência de turno.	Detalhes de HTTP de provedores.
@mpp/providers	Adapters de APIs externas sob interfaces comuns.	Regras de negócio MPP.
@mpp/db	Cliente e contrato TypeScript do banco.	Orquestração conversacional.
@mpp/ingest-functions	Coordenação durável do ciclo de vida das mensagens e jobs.	Duplicação das fórmulas do core.
@mpp/admin	Interface e comandos administrativos, além do endpoint Inngest.	Interface de paciente.

Da bolha do WhatsApp ao evento processável

Fluxograma 2. Entrada, idempotência, buffer e despacho

1. Meta

Entrega webhook de mensagem ou atualização de status.



2. Edge

Valida HMAC SHA-256 e normaliza o payload.



3. RPC atômica

`ingest_whatsapp_inbound` deduplica e persiste.

4. Buffer

Texto e mídia entram em `message_buffer` com `flush_after`.



5. Evento atrasado

A Edge envia `buffer.flush` ao Inngest após debounce e margem.



6. Claim

Worker reivindica o burst e usa `message_dispatch_outbox`.

7. Agregação

Junta textos e preserva IDs, timestamps e mídias.



8. Evento

Dispara `message.received` com o último ID para reply.



9. Serialização

`process-message` roda com concorrência 1 por usuário.

O debounce era configurável em `global_config`, com fallback de 8 segundos. O evento inicial era agendado com margem adicional para reduzir a corrida entre a segunda mensagem e o flush.

Por que havia buffer

Pacientes frequentemente descreviam uma única intenção em várias bolhas: foto, legenda, quantidade e correção. Sem buffer, cada bolha poderia gerar uma chamada separada à LLM e registros parciais. O sistema agrupava o burst e enviava uma única representação ao pipeline.

Dados preservados no burst

todos os `providerMessageIds`

timestamps do provider

texto agregado

tipo de cada mídia

media IDs

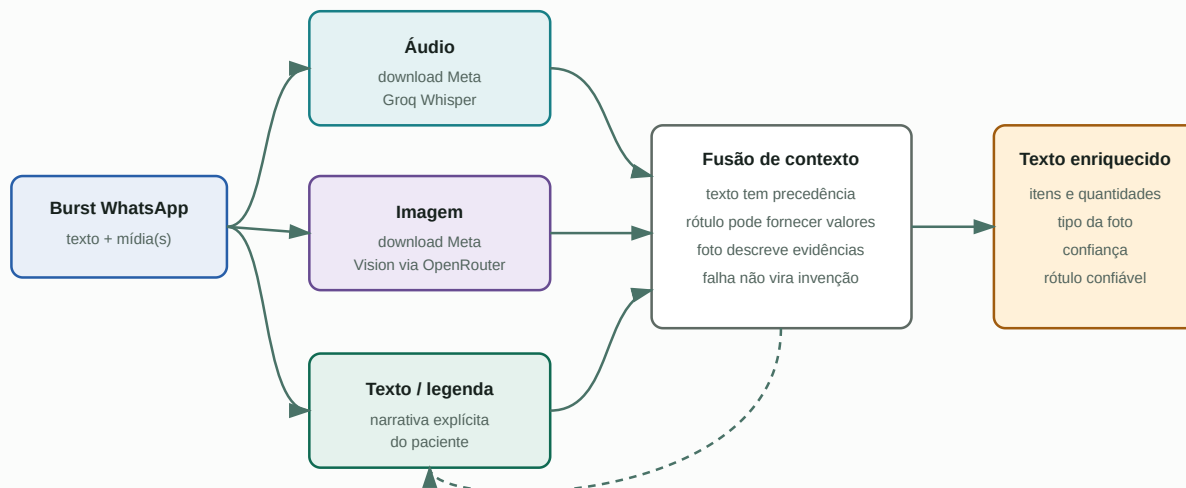
último ID para resposta

Botões eram exceção intencional

Um tap em botão ou lista não aguardava o debounce. A Edge emitia imediatamente `interactive.button.tapped`, pois confirmar ou editar uma proposta é uma ação discreta e determinística.

Áudio, imagem, legenda e contexto

Fluxograma 3. Preparação de mídia antes da LLM



A imagem não era registrada diretamente. Ela era transformada em sinais estruturados e passava pelo mesmo pipeline de validação e confirmação. A legenda era tratada como contexto da foto, não como uma segunda refeição.

Áudio

1. Resolve o media ID na Graph API.
2. Baixa os bytes com retries para falhas transitórias.
3. Transcreve com Groq Whisper.
4. Persiste a transcrição na mensagem inbound.
5. Emite `stt.transcribed` ou `stt.failed`.

Imagem

1. Baixa da Meta e converte para data URI.
2. Classifica refeição, corpo, balança, rótulo ou outro.
3. Extrai sinais e confiança.
4. Aplica política específica para rótulo nutricional.
5. Emite `vision.analyzed` ou falha observável.

Proteção contra burst dividido

Se uma legenda chegasse enquanto STT ou vision ainda estava em voo, o `buffer-listener` podia estender o flush uma única vez. Isso reduzia o risco de processar foto e explicação como turnos independentes.

Pipeline do agente

Fluxograma 4. Decisão conversacional e execução de tools	
Contexto	Garante usuário, checa assinatura, carrega perfil, progresso, snapshot, histórico filtrado, pending, timezone e resumo.
Estágio	Resolve onboarding ou protocolo ativo; carrega prompt e regras do banco no idioma do paciente.
Roteamento	Mantém modelo principal para turnos complexos; permite modelo barato em saudações, status e medições simples.
Prompt	Bloco estável com persona/regras, bloco variável com estado atual, RAG do método e proteção antirrepetição.
Loop LLM	Tool calling limitado por <code>max_tool_iterations</code> , schemas Zod e lista de tools permitidas por estágio.
Guards	Bloqueia escrita falsa, duplicidade espúria, item fantasma, substituição fraca e números inconsistentes.
Saída	Composição determinística, card canônico, comentário educativo, interação ou texto/ áudio humanizado.

Separação de responsabilidade

A LLM PODIA	A LLM NÃO ERA FONTE DE VERDADE PARA
Interpretar texto, áudio transcrito e descrição visual.	Meta calórica, proteína alvo, balanço e Bloco 7700.
Escolher uma tool e preencher argumentos candidatos.	Confirmação de que uma gravação ocorreu.
Gerar prosa de coaching e responder perguntas abertas.	Valores oficiais de alimento quando havia referência canônica.
Gerar dieta e treino quando solicitado.	Idempotência, substituição destrutiva e transações.
Selecionar tom, idioma e formato.	Estado diário persistido e status de assinatura.

Tools funcionais

<code>cadastra_dados_iniciais</code>	<code>define_protocolo</code>	<code>define_meta_peso</code>	<code>registra_refeicao</code>
<code>registra_treino</code>	<code>marca_refeicao_pulada</code>	<code>consulta_progresso</code>	<code>consulta_metricas</code>

consulta_resumo_periodo

gera_dieta

gera_treino

confirma_pais_residencia

pausar_agente

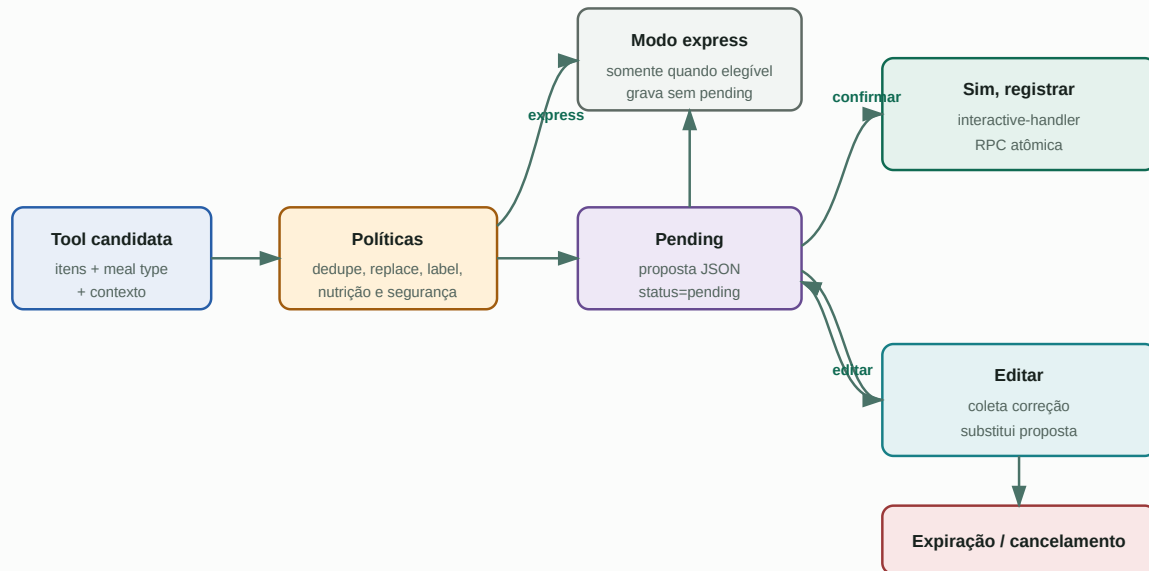
retomar_agente

encerra_atendimento

delete_user

Proposta, confirmação e escrita atômica

Fluxograma 5. Refeição com confirmação interativa



O registro em `meal_logs` não era considerado concluído porque a LLM disse “registrado”. A conclusão exigia uma tool bem-sucedida ou, quando aplicável, o tap do paciente. O handler interativo chamava as mesmas políticas de nutrição e as RPCs de escrita.

Redundâncias dessa etapa

- **Validação Zod:** rejeitava payloads malformados antes da execução.
- **Proveniência nutricional:** ligava o item a `food_db_id` quando havia referência canônica.
- **Replace escopado:** substituição exigia intenção ou overlap suficiente e alvo explícito.
- **RPC atômica:** log e snapshot eram alterados dentro da mesma operação.
- **Idempotência:** provider message ID e request ID impediam duplicação em retries.
- **Composição pós-banco:** tabela e card eram gerados com o estado realmente persistido.

Da resposta validada ao WhatsApp

1. Composição

Texto final, interação ou áudio; card canônico e partes do registro.



2. Humanização

Typing, delays, divisão por bolha e resposta à mensagem correta.



3. Meta API

Envia texto, botão, lista, áudio, imagem, template ou reação.

4. Persistência

Uma linha outbound por entrega, com ID real e status.



5. Status webhook

`sent`, `delivered`, `read` ou `failed`.



6. Auditoria

Custos, tokens, latência, tools e eventos ficam correlacionáveis.

Formatos de saída

FORMATO	QUANDO	DETALHE
Texto único	Resposta simples ou status.	Enviado com typing e delay configurável.
Três bolhas	Registro de refeição.	Tabela da refeição, comentário educativo e card de saldo.
Botões	Até 3 escolhas.	Confirmação, edição ou onboarding.
Lista	De 4 a 10 escolhas.	Menu interativo nativo do WhatsApp.
Áudio	Mensagem-âncora ou preferência de voz.	Texto é reescrito para fala, sintetizado, enviado e auditado.
Template HSM	Contato fora da janela permitida.	Usa template previamente aprovado pela Meta.

Motor determinístico

A função da LLM era entender o relato. As contas oficiais saíam de `@mpp/core`, das referências nutricionais e do estado do banco.

Meta Recomposição: $BMR \times$ multiplicador menos déficit. Ganho: $TDEE \times$ superávit. Manutenção: TDEE.	Saldo Comida: consumido – meta. Exercício não libera mais comida no card.	7700 Net: consumido – meta – exercício. O crédito usa déficit planejado menos saldo líquido.
--	--	---

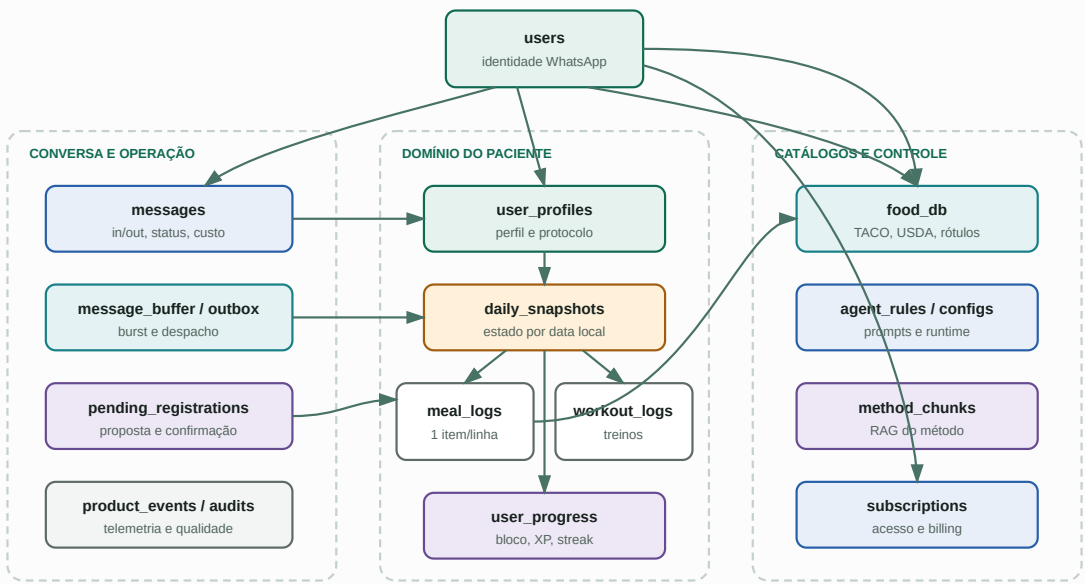


Regras importantes

- Exercício acelera o bloco, mas não aumenta a linha “Restam” no card.
- Dia com gap aberto e sem resposta fecha como incompleto e credita zero.
- Dia explicitamente pulado pode usar o consumo real das demais refeições.
- Consumo abaixo de 50% da meta ativa defesa contra sub-registro.
- Créditos negativos podem reduzir o acumulado, mas o total não fica abaixo de zero.

Arquitetura de dados

Fluxograma 7. Relacionamentos lógicos do domínio



GRUPO	TABELAS PRINCIPAIS	RESPONSABILIDADE
Identidade	users , user_profiles , user_progress	Paciente WhatsApp, perfil clínico, protocolo e gamificação.
Dia	daily_snapshots , meal_logs , workout_logs , reevaluations	Estado por data local e registros confirmados.
Conversa	messages , processed_messages , message_embeddings	Histórico, idempotência, busca e custos.
Entrega	message_buffer , message_dispatch_outbox , tabelas de attempts	Debounce, claim, retries e fechamento de entrega.
Pendências	pending_registrations , pending_approvals	Confirmação do paciente e aprovação interna.
Nutrição	food_db , user_food_corrections , frases educativas	Referência canônica, personalização e comentários.
Agente	agent_configs , agent_rules , versões, global_config , calc_config	Modelos, prompts, regras, flags e parâmetros.
Conhecimento	method_chunks	RAG do método com embeddings e filtro por protocolo.
Billing	subscriptions , subscription_events	Status de acesso e processamento idempotente de webhooks.
Observabilidade	product_events , tools_audit , llm_evaluations , audit_log	Telemetria, rastreabilidade e qualidade.

Mídia não residia em um bucket Supabase

A auditoria live de 02/07/2026 não encontrou buckets de Storage. O sistema persistia media IDs/URLs e baixava os bytes pela Graph API quando necessário. Isso distingue a arquitetura real do resumo genérico do README, que mencionava Storage como capacidade do Supabase.

Workers e automações

O snapshot exportava 17 funções Inngest. Algumas respondiam a eventos emitidos pela Edge ou por pg_cron; outras usavam cron nativo do Inngest. Jobs horários revalidavam o horário local do paciente antes de agir.

WORKER	TRIGGER	FUNÇÃO
process-message	message.received	Pipeline principal, mídia, agente e resposta.
buffer-listener	buffer.flush	Claim, agregação e despacho do burst.
interactive-handler	interactive.button.tapped	Confirma, edita ou processa onboarding interativo.
daily-closer	day.close.tick	Fecha o dia local e atualiza progresso.
daily-gap-checker	day.close.tick	Detecta refeição esperada faltante antes do fechamento.
meal-gap-reminder	cron horário	Lembrete opt-in determinístico.
engagement-sender	engagement.tick	Mensagem proativa com dados reais.
training-daily-delivery	cron horário + gate local	Entrega treino do dia sem nova geração.
pending-cleanup	cron a cada 5 min	Expira propostas vencidas.
daily-audit	cron 5 vezes/dia BRT	Saúde, integridade e auto-fix limitado.
sample-judge	cron 2 vezes/dia BRT	Amostra respostas para LLM-as-judge.
food-db-gaps-report	cron semanal	Reporta alimentos que caíram em fallback.
pipeline-health	pipeline.health.tick	Detecta inbound sem outbound e tenta ressincronizar.
wa-quality-check	wa.quality.check	Monitora qualidade do número Meta.
openrouter-balance-check	cron 4 vezes/dia BRT	Monitora saldo do provedor de IA.
regression-beacon	cron horário	Detecta regressões por eventos operacionais.
user-deletion-purger	cron a cada 15 min	Conclui expurgo de usuários marcados.

Fluxograma 8. Relógios e execução local

pg_cron

Varre em UTC e usa pg_net para emitir ticks.



Inngest

Recebe evento, aplica retries e concorrência.



Gate local

Converte timezone do usuário e decide agir.

Cron Inngest

Dispara jobs de saúde e limpeza.



Config runtime

Janelas, slots e políticas vêm do banco.



Entrega

Usa o mesmo provider WhatsApp e persiste resultado.

Painel administrativo e intervenção

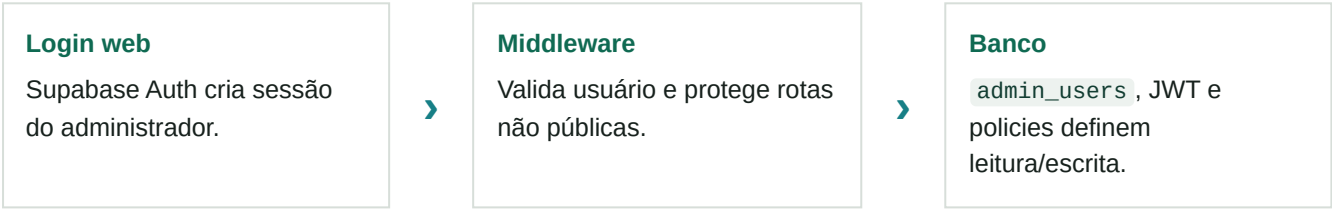
O painel era uma aplicação Next.js hospedada na Vercel. Ele não substituía o canal do paciente; servia para observar, configurar e operar o agente.

Operação
Dashboard, atenção, usuários, ficha, pausa, fechamento manual, conversas, busca, mídia e envio administrativo.

Agente
Prompts versionados, regras, playground, modelos, tools, alimentos, fórmulas e configurações globais.

Saúde e negócio
Auditoria, avaliações, funil, receita, assinaturas, crons e credenciais de serviços.

Fluxo de autenticação administrativa

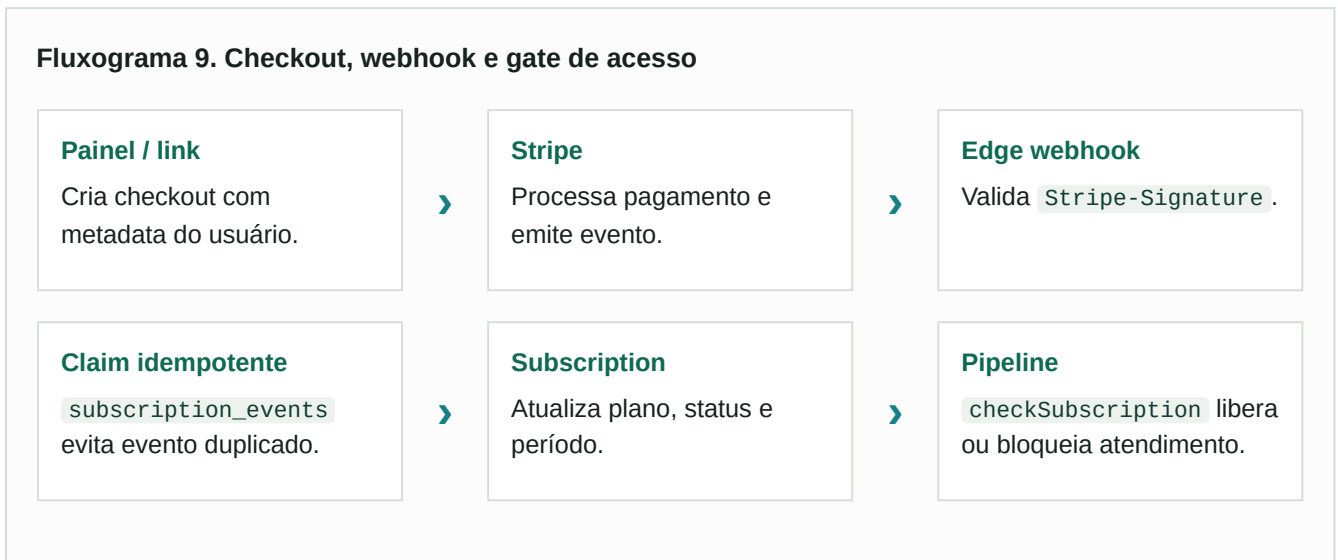


Endpoints hospedados no admin

ENDPOINT	PAPEL
/api/inngest	Serve e sincroniza a coleção de functions Inngest.
/api/admin/send-message	Envio manual pelo provider com autenticação própria.
/api/media/[id]	Proxy autenticado para visualizar mídia da Meta.
/api/stripe/checkout	Cria checkout associado ao usuário do domínio.
/api/stripe/setup-products	Operação de catálogo de billing.

Assinaturas e controle de acesso

Fluxograma 9. Checkout, webhook e gate de acesso



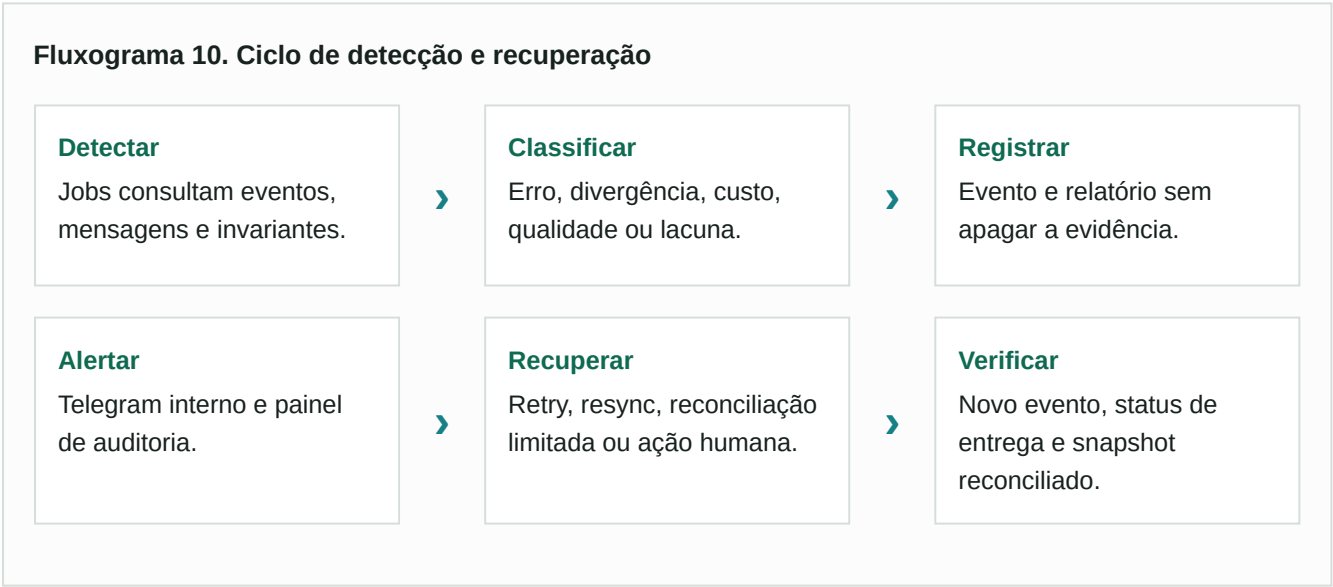
O webhook tratava checkout concluído, criação/atualização/cancelamento de assinatura, pagamento bem-sucedido e pagamento falho. O vínculo priorizava `user_id` explícito e possuía fallbacks históricos por WhatsApp ou e-mail.

Billing era suporte ao produto WhatsApp

Stripe não era uma interface de paciente comparável ao app. Seu estado apenas determinava se o pipeline conversacional podia atender.

Observabilidade, auditoria e recuperação

CAMADA	EVIDÊNCIA PRODUZIDA	RESPOSTA
Mensagem	Inbound/outbound, provider IDs, status, latência e modelo.	Reprocessamento, inspeção e correlação.
Tool	<code>tools_audit</code> com nome, args, resultado e sucesso.	Prova de escrita real e análise de falha.
Produto	<code>product_events</code> para guards, pipeline, mídia, custos e crons.	Alertas e painéis.
Qualidade LLM	<code>llm_evaluations</code> e sample judge.	Amostragem, nota e regressão.
Integridade	Daily audit compara snapshots, logs e bloco.	Alerta e auto-fix restrito com circuit breaker.
Infra	Qualidade Meta, saldo OpenRouter, pipeline sem resposta.	Telegram e tentativa de recuperação.
Admin	<code>audit_log</code> e histórico imutável de configs/regras.	Rastreabilidade de alteração humana.



Segurança e autorização na arquitetura

FRONTEIRA	CONTROLE ESPERADO NO FLUXO	PRIVILÉGIO
Meta → Edge WhatsApp	Challenge de verificação e HMAC SHA-256 sobre corpo bruto.	Endpoint público com validação interna.
Stripe → Edge billing	<code>Stripe-Signature</code> e secret do webhook.	Endpoint público com service role após validação.
Telegram → Edge	Secret token do bot e controle de admin.	Operação interna.
Inngest → Vercel	Signing key e protocolo da biblioteca.	Execução de workers server-side.
Admin → Supabase	Sessão Auth, middleware, <code>admin_users</code> , RLS e roles.	JWT do administrador.
Edge/workers → Supabase	Service role, sem exposição ao cliente.	Bypass de RLS para operações de backend.
Providers externos	Secrets em env ou <code>service_credentials</code> .	Somente server-side.
Paciente	Número WhatsApp é identidade de domínio.	Não recebe acesso direto ao PostgREST.

Edge Functions com `verify_jwt=false`

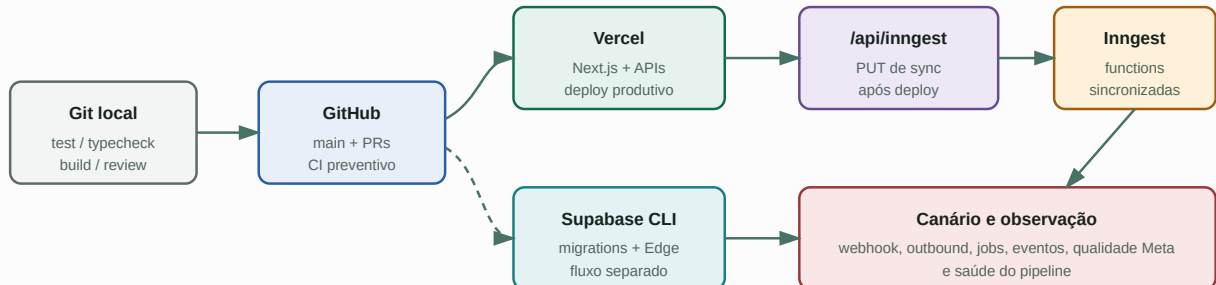
Os webhooks eram públicos por natureza e faziam autenticação no próprio handler. Isso não equivale a ausência de autenticação, mas torna a correção de cada verificação interna parte do perímetro crítico.

Tratamento de secrets

O sistema previa variáveis por categoria: Supabase, Meta, OpenRouter, Groq, TTS, Inngest, Stripe, Telegram, observabilidade e e-mail. Valores não pertenciam ao repositório. Alguns serviços também podiam ser configurados em `service_credentials`, acessada apenas pelo backend e pelo painel administrativo autorizado.

DevOps, ambientes e deploy

Fluxograma 11. Caminho de código até produção na fase WhatsApp



Características do fluxo

- Vercel hospedava o admin e o endpoint Inngest; não havia Vercel Cron.
- Supabase migrations e Edge Functions tinham ciclo operacional separado.
- `scripts/deploy.sh` executava deploy Vercel, aguardava propagação e forçava `PUT /api/inngest`.
- O script abortava se a sincronização do Inngest não retornasse sucesso.
- O workflow GitHub existente tinha escopo restrito: impedir novos usos não justificados de `as any` no agent.
- Scripts de seed, dry-run, reparo e auditoria serviam como ferramentas operacionais auditáveis.

Redundâncias e modos de falha

RISCO OPERACIONAL	REDUNDÂNCIAS IMPLEMENTADAS	RESULTADO ESPERADO
Meta reenvia webhook	Índices únicos por provider ID e RPC de ingestão idempotente.	Uma mensagem lógica, mesmo com entrega repetida.
Paciente manda várias bolhas	Debounce, burst com todos os IDs e filtro do contexto atual.	Uma intenção, sem duplicar a mensagem na LLM.
Worker cai após claim	Outbox de despacho, completion explícita e retries Inngest.	Recuperação sem perder o burst.
Foto e legenda se separam	Gate de mídia em voo e extensão única do buffer.	Mais chance de analisar como um único turno.
LLM afirma que salvou	Fake-write detector, tool obrigatória e card pós-banco.	Sem confirmação falsa de escrita.
LLM tenta substituir refeição	Intenção explícita, overlap, adição detector e RPC escopada.	Correção altera só o alvo.
Valor nutricional inconsistente	Catálogo verificado, <code>food_db_id</code> , invariantes e confirmação.	Número ligado à referência ou marcado como estimativa.
Resposta externa falha	Retries, fallback ao paciente, delivery status e evento de erro.	Falha visível e recuperável.
Deploy deixa workers antigos	Sync explícito do Inngest e pipeline-health.	Detecção e correção de dessincronização.
Job age no dia errado	Timezone IANA, timestamps do provider e gate pela data local.	Fechamento e refeição no dia do paciente.

Sem promessa de “exactly once” na rede

Meta, Vercel, Inngest e Supabase formavam um sistema distribuído. A garantia prática vinha da combinação de entrega pelo menos uma vez, chaves idempotentes, claims, transações, concorrência por usuário e reconciliação. Isso é mais realista do que depender de uma única chamada perfeita.

Limitações e caveats do período

Os pontos abaixo ajudam a interpretar a arquitetura sem projetar sobre ela capacidades que vieram depois.

Sem identidade portátil de paciente

O número WhatsApp era a chave operacional. Não havia login de paciente, sessão própria, recuperação de conta ou vínculo independente do canal.

Mídia dependente da Meta

Sem bucket próprio, o backend precisava resolver e baixar media IDs da Graph API no momento do processamento ou visualização.

Configuração distribuída

Comportamento podia vir de env, `service_credentials`, prompts, rules e configs de runtime. Isso dava flexibilidade, mas exigia inventário e auditoria para saber a configuração efetiva.

Dois relógios

Parte dos jobs nascia no pg_cron e parte no Inngest. A data correta dependia do gate pelo timezone do paciente.

Achados da auditoria de 02/07/2026

A auditoria registrou exposição excessiva em grants/RLS/RPCs/views, dependências com advisories, ausência de índice adequado em um update de status WhatsApp e riscos em fetchs externos. Migrations posteriores podem ter alterado parte desse cenário. Este relatório não reexecutou a auditoria live e, portanto, não afirma o estado atual desses achados.

Divergências documentais corrigidas neste relatório

- O README e documentos mais antigos citavam modelos e contagens anteriores. O modelo era runtime-configurável.
- No baseline havia **114 migrations**, **6 Edge Functions** e **17 funções Inngest**.
- Havia um workflow GitHub limitado; não era uma suíte CI completa.
- Supabase Storage era capacidade da plataforma, mas a auditoria live não encontrou bucket configurado.

Como tudo se interligava

1. O paciente enviava texto, áudio, foto ou tap no WhatsApp.
2. A Meta chamava a Edge Function, que validava assinatura e persistia o evento.
3. Mensagens livres entravam em buffer; taps seguiam direto ao handler determinístico.
4. Inngest serializava por usuário, recuperava bursts e coordenava retries.
5. Áudio era transcrito; imagem era analisada; legenda e contexto eram fundidos.
6. O agent carregava banco, prompt, timezone, assinatura e estado diário.
7. A LLM escolhia tools; guards e Zod validavam a intenção.
8. Core, catálogo nutricional e RPCs realizavam as contas e escritas.
9. O sistema reconstruía tabela, comentário e card a partir do banco.
10. O provider enviava pela Meta e persistia IDs/status reais de cada bolha.
11. Crons fechavam dias, lembravam gaps, enviavam treino e monitoravam saúde.
12. O painel Next.js permitia observar e administrar; Stripe controlava acesso; Telegram recebia alertas.
13. Vercel, Supabase e Inngest formavam os três pilares de execução, estado e orquestração.

Resumo arquitetural em uma frase

Era um SaaS conversacional orientado a eventos: WhatsApp como interface, Supabase como sistema de registro, Inngest como coordenador, Vercel como runtime web, providers de IA como interpretação e o core determinístico como dono da verdade.

Evidências técnicas consultadas

Os caminhos abaixo referem-se ao repositório no snapshot `280b3d7`. A lista prioriza arquivos que demonstram cada conexão descrita.

- `package.json`, `pnpm-workspace.yaml`, `turbo.json`: runtime, monorepo e scripts.
- `apps/admin/package.json`: Next.js, React, Inngest, Stripe e Supabase.
- `apps/admin/src/app/api/inngest/route.ts`: exposição das functions na Vercel.
- `apps/admin/src/middleware.ts`: sessão administrativa e rotas públicas específicas.
- `supabase/functions/webhook-whatsapp/index.ts`: assinatura Meta, ingestão, buffer e taps.
- `packages/providers/src/messaging/whatsapp-cloud.ts`: Graph API, mídia, botões, listas, reações e status.
- `packages/inngest-functions/src/functions/buffer-listener.ts`: claim, outbox, burst e gate de mídia em voo.
- `packages/inngest-functions/src/functions/process-message.ts`: STT, vision, agent, envio e persistência.
- `packages/inngest-functions/src/index.ts` e `client.ts`: 17 workers e eventos tipados.
- `packages/agent/src/pipeline.ts`: contexto, assinatura, prompts, RAG, roteamento, tools e guards.
- `packages/agent/src/tools.ts`: catálogo de tools e escritas de domínio.
- `packages/core/src/engine/targets.ts`, `balance.ts`, `bloco.ts`: fórmulas oficiais.
- `packages/providers/src/llm/openrouter.ts`: gateway, timeout, retries, cache e custo.
- `supabase/migrations/`: 114 migrations, schema, RLS, RPCs, outboxes e invariantes.
- `supabase/functions/webhook-stripe/index.ts`: assinatura e processamento idempotente de billing.
- `scripts/deploy.sh`: Vercel, sync Inngest e gate de sucesso.
- `.github/workflows/lint-agent.yml`: proteção incremental da qualidade.
- `docs/adr/`, `docs/CALCULO-MPP.md` e runbooks: decisões e operação.
- `docs/audits/2026-07-02-full-platform-audit.md`: validações live e caveats históricos.

Contagens do baseline

114 Migrations Supabase versionadas.	6 Edge Functions no repositório.	17 Funções Inngest exportadas.
--	--	--

Glossário

TERMO	SIGNIFICADO NO SISTEMA
Agente MPP	Nome técnico do produto conversacional que posteriormente passou a compor a visão BodyFlow.
Burst	Conjunto de bolhas do paciente agrupadas como um único turno.
Pending	Proposta aguardando confirmação ou edição antes da escrita.
Tool	Operação tipada que a LLM solicita e o sistema valida/executa.
Snapshot	Estado agregado de um usuário em uma data local.
Bloco 7700	Acumulador de déficit líquido usado na progressão do método.
RAG	Recuperação de trechos do método por similaridade vetorial.
Outbox	Registro durável intermediário para garantir despacho recuperável.
Idempotência	Repetir a mesma tentativa não deve criar um segundo efeito lógico.
Service role	Credencial server-side do Supabase com privilégios elevados.
HSM	Template de mensagem aprovado pela Meta para contato fora da janela.
Gate local	Decisão de um job baseada no timezone e na data/hora do paciente.

Nota final

Este é um documento descritivo e histórico. Ele não modifica código, banco ou produção e não representa a arquitetura futura do app nativo.